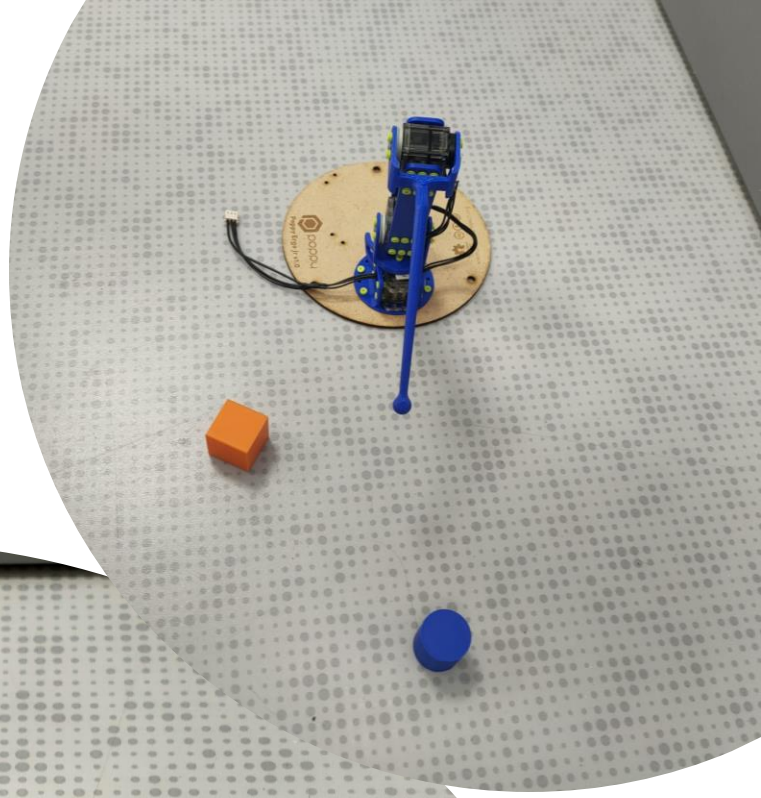
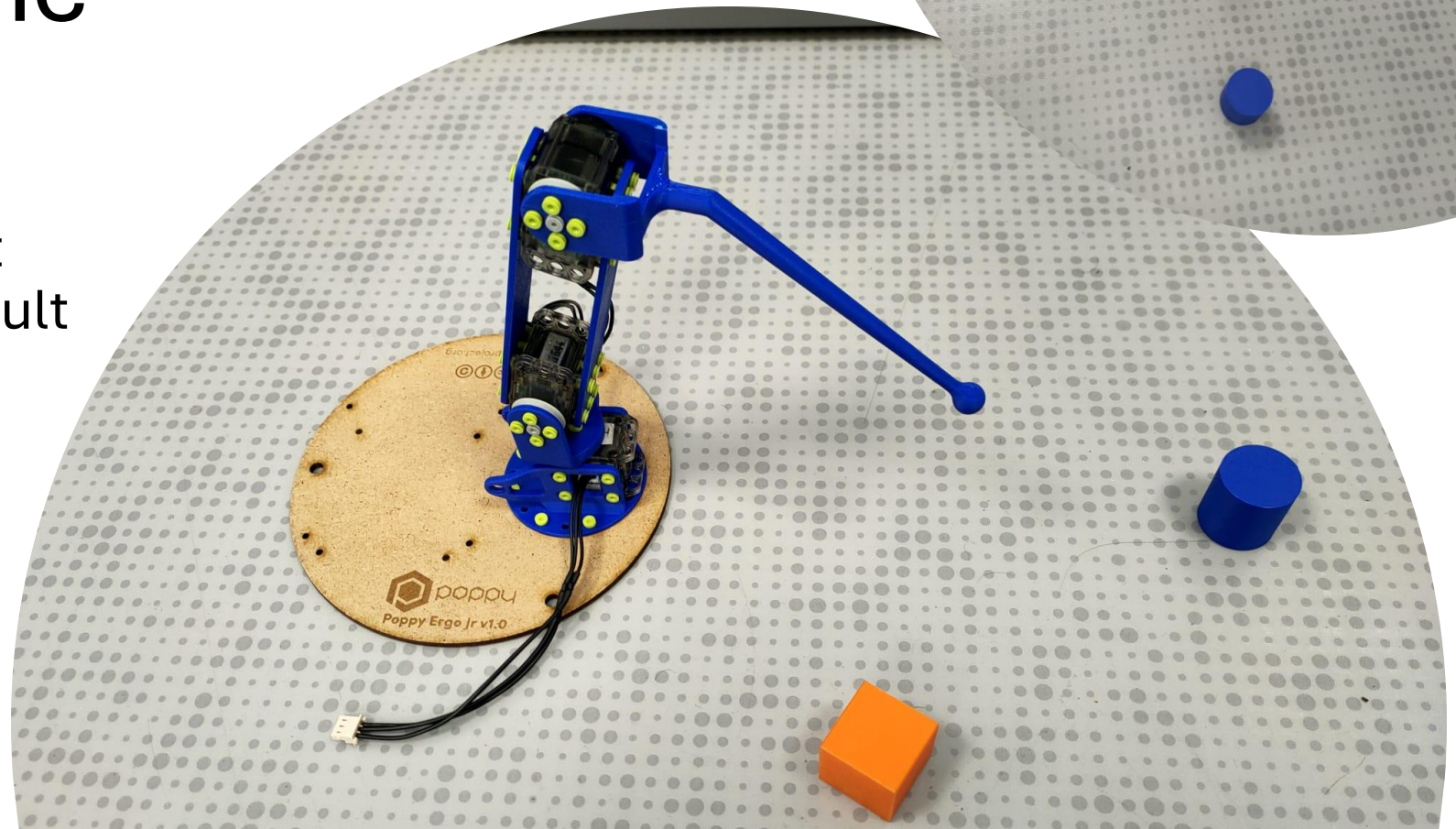
Informatique / iut de BORDEAUX

# S6.A.01 Évolution d'une application

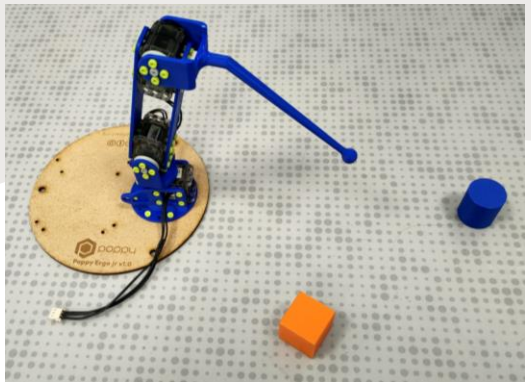
Mélodie Daniel, Clément Gaspard, Grégoire Passault & Arnaud Pêcher

2025 - 2026





# Sujet de la SAE S6.A.01



- **La SAE S6.A.01 prolonge la SAE S5.A.01-7** en réutilisant le modèle YOLO et la chaîne de localisation 3D (calibration intrinsèque, calibration extrinsèque, transformation vers le repère monde) pour détecter un cube et un cylindre et transmettre leurs positions à un bras robotique custom. Le robot doit réaliser une tâche de **Push-in-Hole** : pousser un objet jusqu'à lui faire atteindre une pose désirée (une position et une orientation). Un agent d'**apprentissage par renforcement (DRL)** contrôle les articulations du bras pour apprendre à faire la tâche, d'abord en simulation. Il faudra ensuite également évaluer l'agent  sur le robot réel.
- **Compétences à acquérir**
  - Prise en main et interfaçage d'un bras robotique.
  - Réutilisation et intégration d'un modèle de détection (YOLO) dans une architecture perception–action complète.
  - Exploitation des calibrations intrinsèque et extrinsèque pour la localisation 3D et la transformation de repères.
  - Utilisation d'un simulateur physique (MuJoCo) pour l'entraînement et la validation des politiques de contrôle.
  - Mise en œuvre d'algorithmes d'apprentissage par renforcement via Stable-Baselines3 et RL-Zoo.
  - Analyse, comparaison et étude des algorithmes de DRL (DDPG, TD3, SAC, etc.) ainsi que de leurs hyperparamètres et des fonctions de récompenses.
  - Conception d'expérimentations et évaluation des performances en simulation et sur le robot réel (passage Sim-to-real).
  - Gestion et organisation d'un projet.

# Application à réaliser

**Spécifications fonctionnelles**

- Matériel fourni : les objets à détecter et à localiser, une caméra (RealSense D435i ou D435if), un ordinateur équipé d'une carte GPU (salle 210) et un bras robotique à assembler (voir le tuto *Assemblage_robot.pdf* sur Moodle).
- Version de la SAE S5.A.01-7 (Localisation par rapport au marker ArUco id 6) :

https://github.com/MelodieDANIEL/sae_s5_a_01_7

- Environnement de simulation : MuJoCo et Gymnasium
  - https://mujoco.readthedocs.io/en/stable/overview.html
  - https://www.gymnasium.ai/
- Algorithmes DRL : Stable baselines 3
  - https://stable-baselines3.readthedocs.io/
  - https://rl-baselines3-zoo.readthedocs.io/
  - https://gregwar.com/rl/
- Exemples de code pour du Pushing:
  - https://github.com/zichunxx/panda_mujoco_gym
  - https://robotics.farama.org/envs/fetch/push/
- Interface graphique : exemples d'outils
  - Streamlit : https://streamlit.io/
  - Gradio : https://www.gradio.app/guides/quickstart

Informatique / iut de BORDEAUX

# Modalités

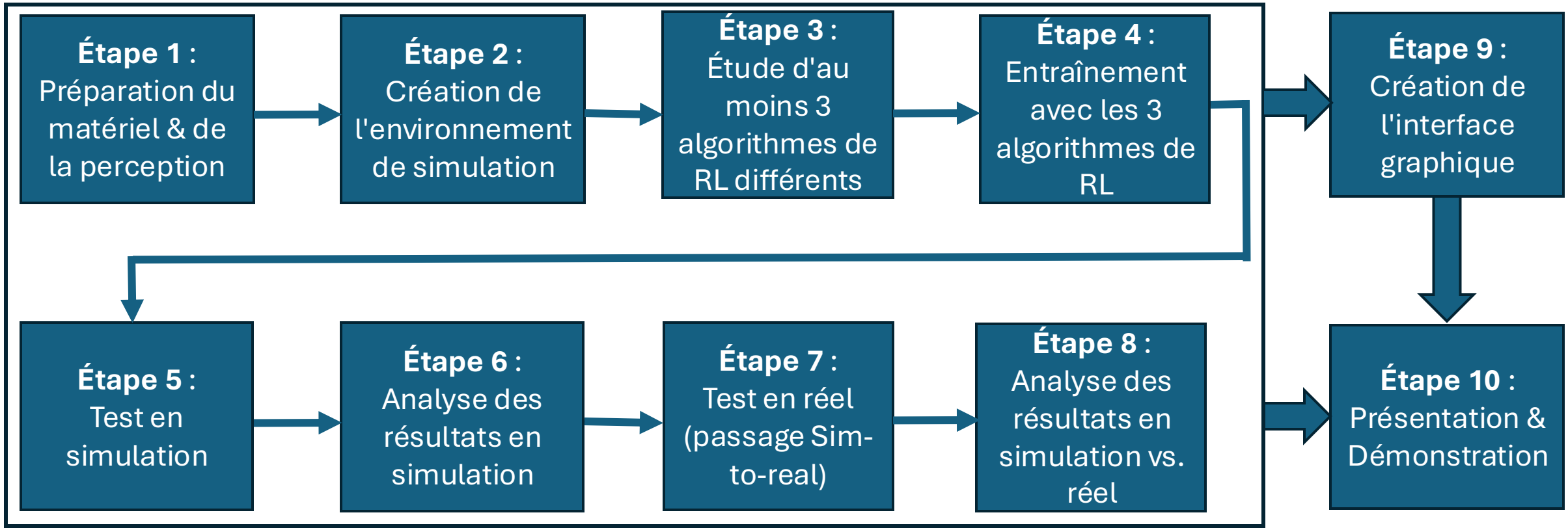
## Organisation

- Par équipe de 5-6 personnes. Les groupes sont disponibles sur Moodle (https://moodle.u-bordeaux.fr/user/index.php?id=25496).
- Désignation d'un responsable du matériel par équipe.
- Planning : sur 5 jours du 16/03/26 au 26/03/26 (~36h dont ~2-3h d'évaluation orale).
- **Conseil** : créer par groupe un git repository pour le projet.

## Évaluation

- 4 Points : Évaluation individuelle (auto-évaluation).
- 16 Points : Présentation et démonstration du projet (pensez à prendre des vidéos au cas où votre projet n'est pas fonctionnel le jour J).

# Déroulement de la SAE



# Étape 1 : Préparation du matériel & de la perception

**Prérequis :** un membre de chaque groupe récupère les objets à détecter et à localiser (un cube orange et un cylindre bleu) ainsi qu'une caméra RealSense D435, D435i ou D435if (https://github.com/realsenseai/librealsense) auprès de votre enseignant(e).

**Rappel :** des PC sous Ubuntu équipés d'une carte GPU sont disponibles dans la salle 210.

**Documents fournis sur Moodle** : le lien vers le dépôt Git (https://github.com/MelodieDANIEL/sae_s5_a_01_7) permet de faire la détection et la localisation des objets en 3D par rapport au repère monde représenté par l'ArUco marker dont l'identifiant est 6.

**Alternative :** vous pouvez aussi utiliser vos codes de la SAE S5.A.01-7.

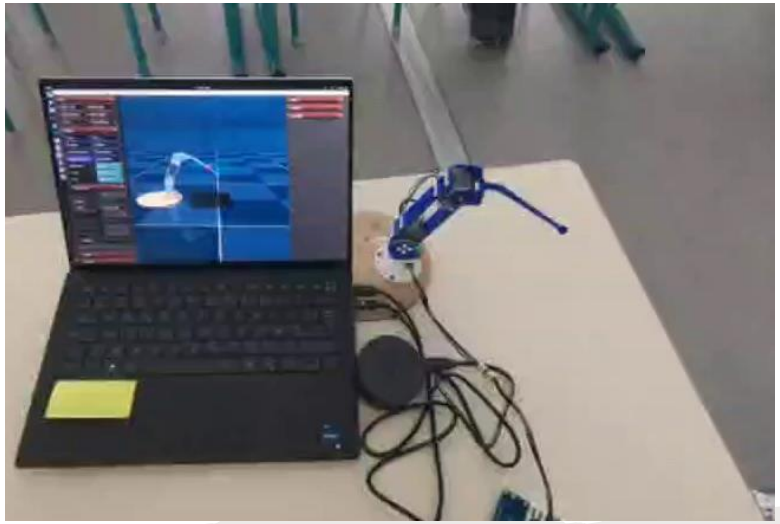
**Remarque :** il faudra probablement refaire les entraînements et la base de données.

**Conseil :** créer un environnement virtuel Python dans lequel vous installerez tous les packages nécessaires.

# Étape 2 : Création de l'environnement de simulation

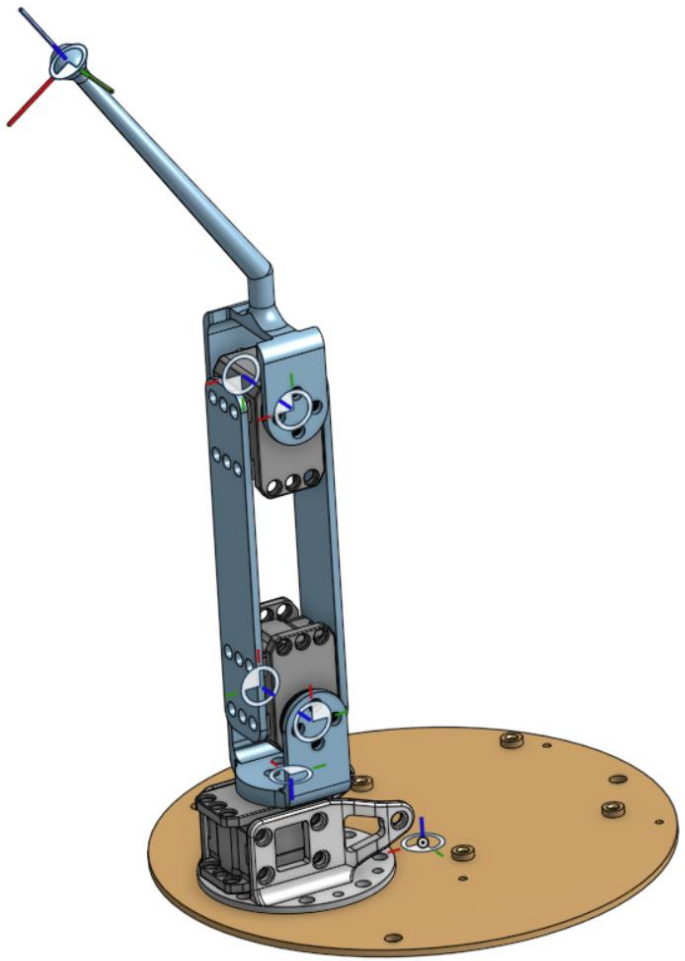
- **Connaissances préalables :**
  - Un bras robotique se compose d'articulations, contrôlées par des moteurs.
  - En modifiant les angles de ces moteurs, le robot bouge.
  - L'objectif est d'arriver à dire quels angles doivent prendre les moteurs pour que l'effecteur final (la boule) atteigne une position 3D désirée dans l'espace.

- **Documents fournis sur Moodle :**
  - Le fichier *scene_push.xml* qui contient la description du robot et des objets (les articulations, les frottements, etc.).
  - La commande *onshape-to-robot-mujoco scene_push.xml* permet de lancer l'environnement de simulation de base sous MuJoCo avec le robot et les objets, puis de les manipuler manuellement à l'aide des sliders en haut à gauche.
  - Le ficher *sim_3dofs.py* contient le code de base pour contrôler le robot dans le simulateur MuJoCo.
  - Le fichier *mirror.py* permet de refaire dans le simulateur ce que fait le robot réel (ce fichier permet de vérifier que le robot a bien été assemblé).



# Étape 2 : Création de l'environnement de simulation

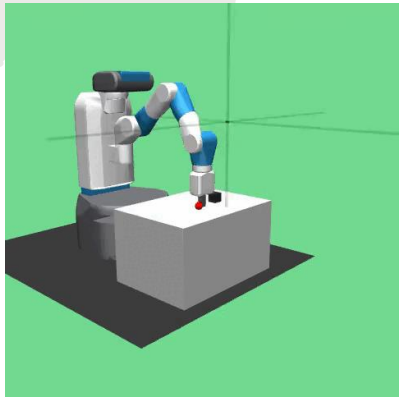
- **Informations complémentaires**
  - **Repère monde** : dans MuJoCo, il est identique au repère de la base du robot. L'origine (0,0,0) se situe au centre de la plaque.
  - **Repère de l'effecteur final** : situé au centre de la boule de l'effecteur final (diamètre = 1 cm).
  - **Limites articulaires du robot** : [−150°,150°] pour chaque moteur, soit [−2.61799, 2.61799] radians pour les limites des actions dans MuJoCo.
  - **Packages à installer** :
    - mujoco
    - stable-baselines3
    - gymnasium
    - numpy
    - sb3-contrib
    - dynamixel-sdk
    - onshape-to-robot
    - pyserial
    - placo



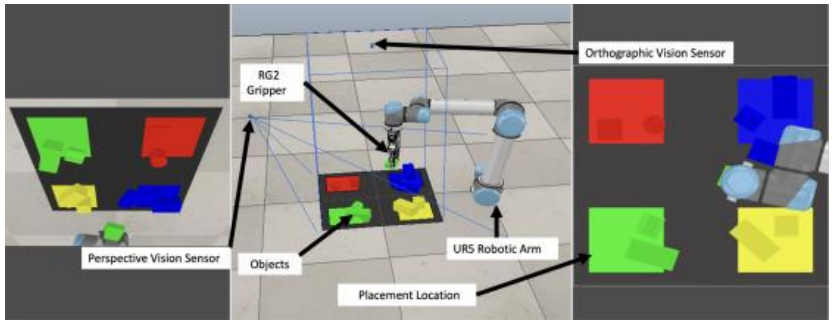
# Étape 2 : Création de l'environnement de simulation

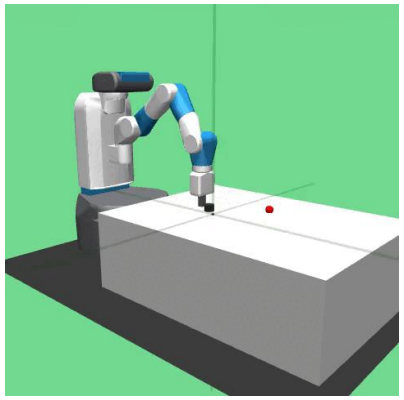
- **Objectif :** Adaptation de l'environnement à Gymnasium.
- **Méthodes principales**
  - __init__()
    - Définition des espaces : observation_space, action_space
    - Initialisation du modèle MuJoCo
  - get_obs ()
    - Construction du vecteur d'observation
    - Exemple : positions articulaires, vitesses, position 3D de l'objet, distance à la cible
  - reset()
    - Réinitialisation de la simulation
    - Retour de l'observation initiale
  - render() : Visualisation de l'environnement
  - step(action)
    - Application de l'action
    - Avancement de la simulation
    - Calcul de la récompense (Reward en anglais)
    - Retour : (observation, reward, terminated, truncated, info)
    - **Remarque sur la reward** : reward = ma_reward + coefficient * action_rate avec action_rate = float(np.sum((action - prev_action) ** 2))
  - set_action(action)
    - Conversion de l'action RL (ex : vitesses, couples, incréments articulaires)
    - Envoi des commandes aux actionneurs du robot

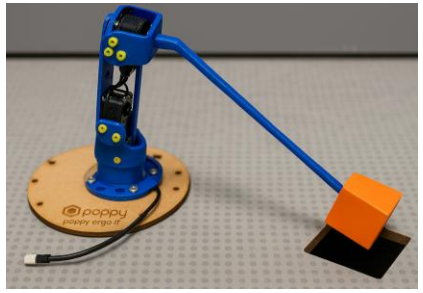
# Étape 2 : Création de l'environnement de simulation



- **Conseil** : Faites des environnements plus simples que le Push-in-Hole pour tester que l'environnement est fonctionnel.
- **Décomposition progressive des tâches**
  - Reaching : L'effecteur atteint une cible 3D (sans interaction complexe).
  - Pushing : Déplacer un objet vers une cible par contact.
  - Sorting : Déplacer les objets pour les trier.
  - Sliding : Cogner un objet pour lui faire atteindre une position désirée.
  - Push-in-Hole : Déplacer un objet pour le faire tomber dans un trou.
- **Documentation :**
  - https://gregwar.com/rl/
  - https://mujoco.readthedocs.io/en/stable/overview.html
  - https://www.gymnasium.ai/
  - https://github.com/zichunxx/panda_mujoco_gym
  - https://robotics.farama.org/envs/fetch/push/





# Étape 3 : Étude d'au moins 3 algorithmes de RL différents

- **Objectif** : comprendre et comparer les différents algorithmes de DRL.
- **Les différents algorithmes DRL**
  - Les architectures Acteur-Critique : DDPG, TD3, SAC, PPO, etc.
  - Les surcouches : CrossQ, HER, etc.
- **Comparaison au niveau des caractéristiques** :
  - On-Policy vs. Off-Policy
  - Le nombre de paramètres.
  - Les fonctions d'activation et de perte choisies.
  - La rapidité d'exécution.
  - Le temps d'entraînement.
  - Les courbes d'apprentissage, etc.
- **Documentation** :
  - https://arxiv.org/
  - https://towardsdatascience.com/
  - https://medium.com/
  - https://wandb.ai/site/
  - https://github.com/optuna/optuna?tab=readme-ov-file
  - https://stable-baselines3.readthedocs.io/
  - https://rl-baselines3-zoo.readthedocs.io/

# Étape 4 : Entraînement avec les 3 algorithmes de RL

- **Prérequis** : les environnements et les 3 algorithmes.
- **Plusieurs paramètres à prendre en compte :**
  - Le taux d'escompte (Discount Factor en anglais).
  - Le taux d'apprentissage (Learning Rate en anglais).
  - Le nombre de timestep par épisode (on coupe l'épisode au-delà et on reset l'environnement).
  - Le nombre d'épisodes ou le nombre global de timesteps .
  - La taille de la mémoire (ReplayBuffer).
  - La taille du batch.
  - Le nombre de mise à jour (calcul de la descente de gradient).
  - L'Early Stopping pour éviter l'Overfitting (ou WandB).
  - Le calcul de la récompense (Reward en anglais).
  - Le seuil (Threshold en anglais).

    .
    .
    .
- **Documentation** : TD3 et TD4 du cours de RL (https://gregwar.com/rl/)

# Étape 5 : Test en simulation & Étape 6 : Analyse des résultats en simulation

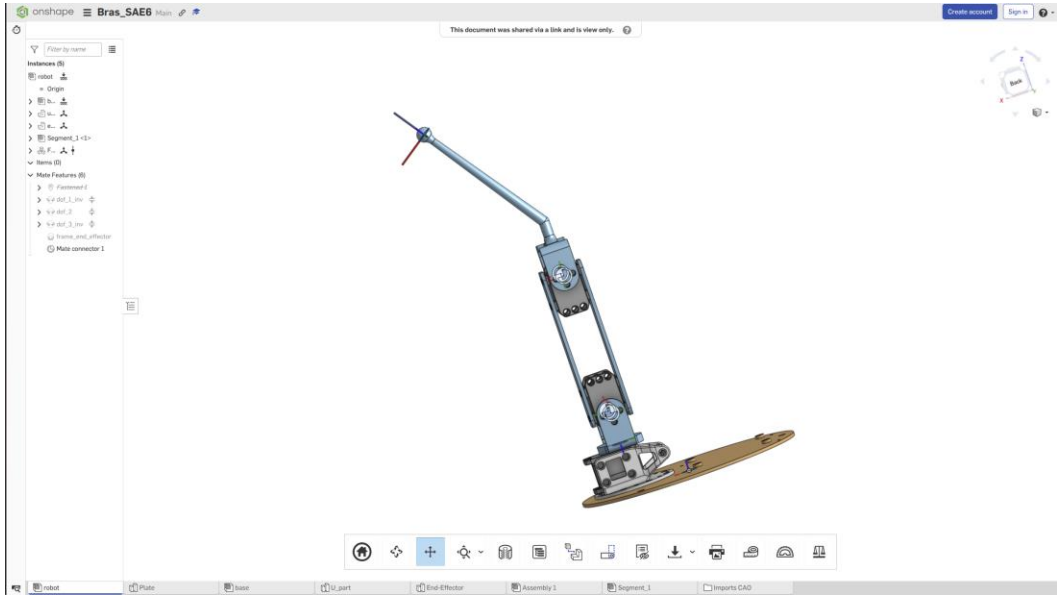
- **Prérequis** : Modèles entraînés et sauvegardés ainsi que les environnements utilisés pour les entraînements.
- **Procédure de test** :
  - Charger les modèles entraînés
  - Les exécuter en **mode test** (sans exploration aléatoire)
  - Évaluer les performances sur plusieurs épisodes
- **Benchmarks :** comparer les performances des différents algorithmes entre eux selon les métriques choisies, et analyser également la différence entre les résultats obtenus lors de l'entraînement et ceux en phase de test.
  - Le taux de succès (ou Success Rate en anglais) : pourcentage du nombre de fois où la tâche est accomplie (info["is_success"] = True).
  - La précision : La moyenne des précisions atteintes par l'agent (va dépendre du threshold).
  - Relever le nombre de timesteps nécessaire en moyenne pour réussir la tâche.
  - Généralisabilité de l'apprentissage

# Étape 7 : Test en réel (passage Sim-to-real)

- Prérequis : Construire le robot.

- Modèle 3D du robot sur OnShape:

https://cad.onshape.com/documents/9cd2d37f10a20c963aa8c3b1/w/751e7ec4f0185e2cc133f396/e/9a387e7c90ee018179d50e3b?renderMode=0&uiState=69b011a664478a7bc2b10edb



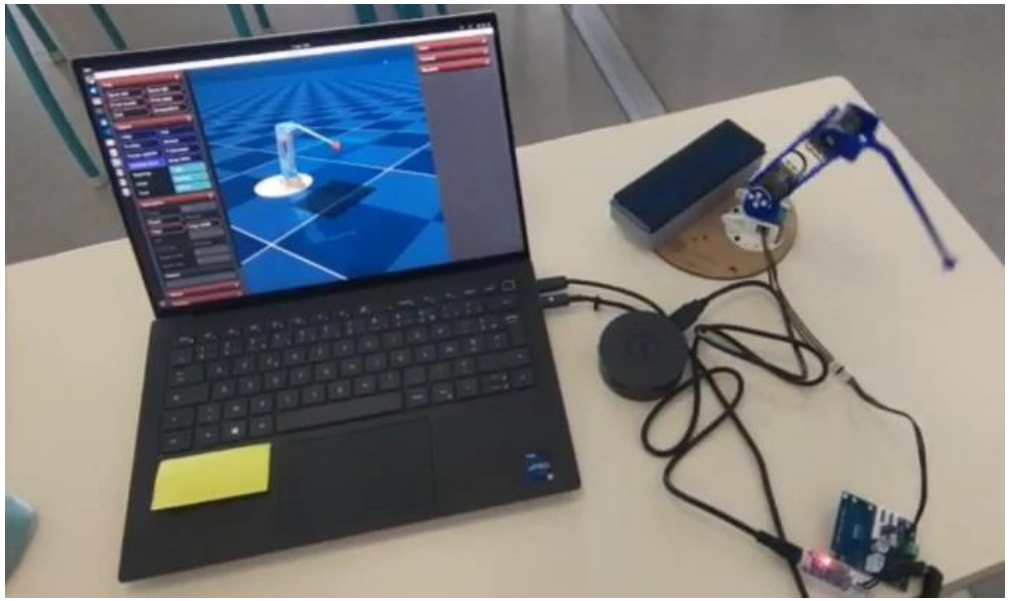

- Tuto d'assemblage disponible sur Moodle : dans le fichier *Assemblage_robot.pdf*.

- Vérification : Pensez à vérifier que le montage est bien réalisé à l'aide du fichier *mirror.py*.

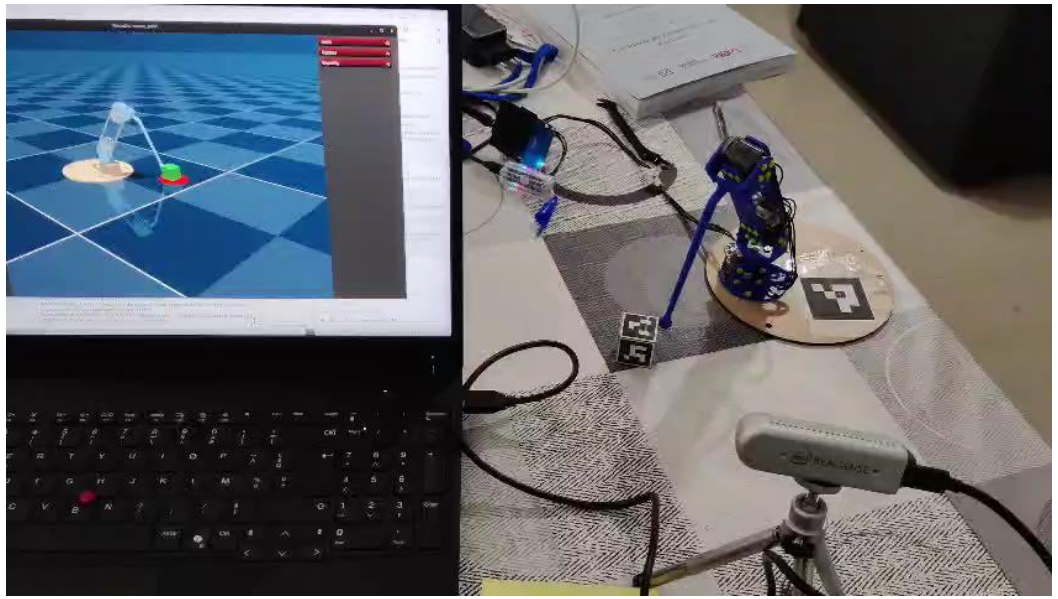
# Étape 7 : Test en réel (passage Sim-to-real)

- **Prérequis** : les modèles entraînés en simulation et le robot assemblé.
- **Objectif** : tester les modèles sur le robot réel.
- **La problématique** : Même si le robot réussit parfaitement la tâche en simulation, transférer directement l'agent sur le robot réel ne devrait pas bien marcher à cause du non-réalisme de la simulation. Il faut ajouter au moment de l'entraînement du bruit de mesure notamment sur (la position des moteurs et la position 0° des moteurs, etc.).
- **Conseil** : Commencer par un environnement simple comme le Reaching avant de passer à des environnements plus complexes.
- **Remarque** : Pour tester sur le robot réel il faut installer la bibliothèque qui permet de réduire les délais de communication avec le robot (à faire une fois).
  - Sudo apt install setserial
  - setserial /dev/ttyUSB0 low_latency ou ACM0 à la place de USB0 (ça dépend des modèles)

# Étape 7 : Test en réel (passage Sim-to-real)



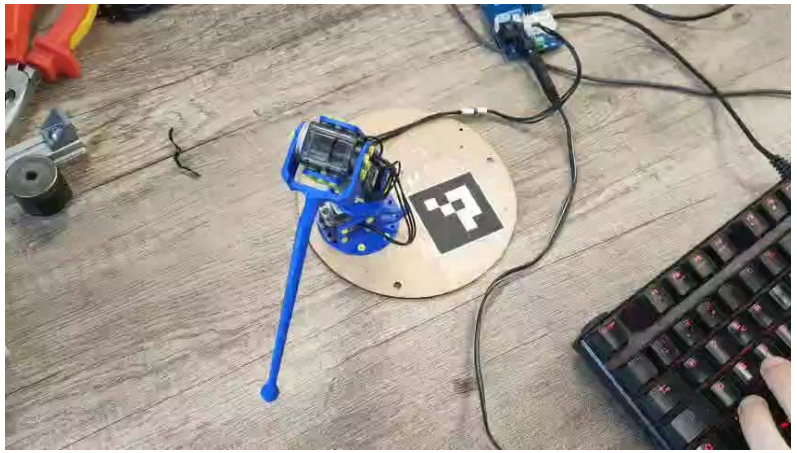
**Reaching**

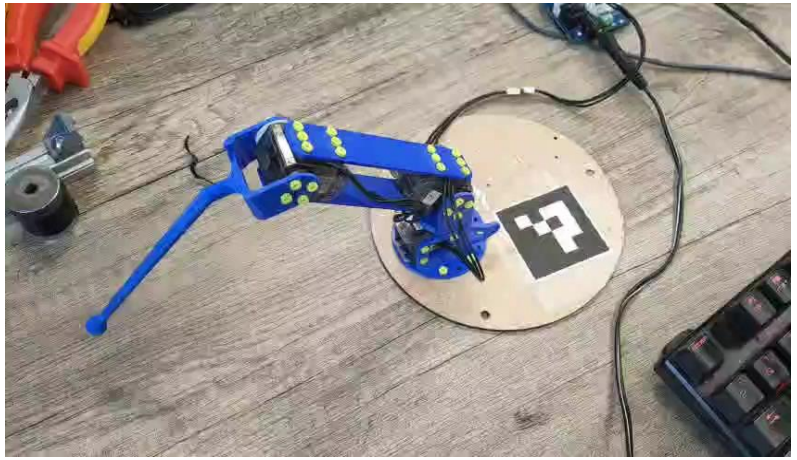


**Push**

# Étape 7 : Test en réel (passage Sim-to-real)

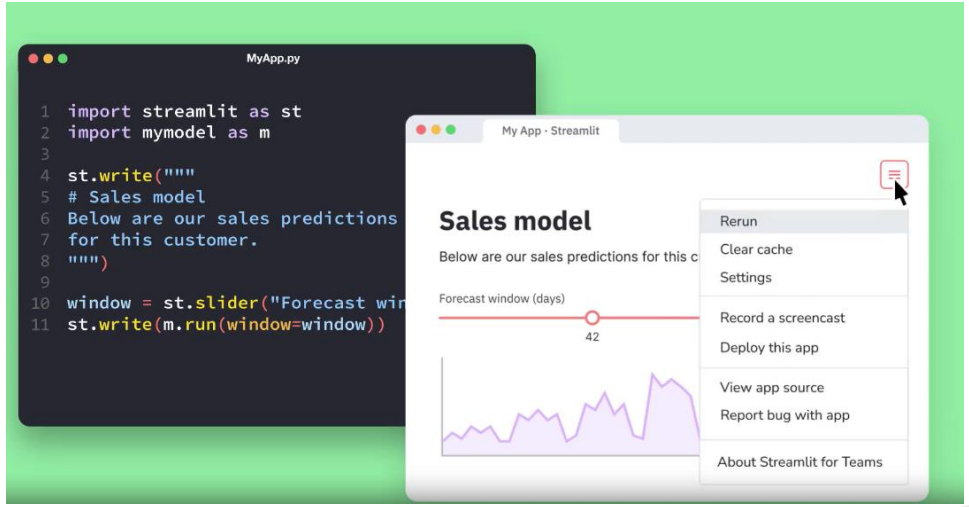
- **Ressources**
  - Le fichier *zero_dxl.py* permet de remettre les moteurs à l'angle 0°.
  - Attention : Ne pas forcer sur les moteurs s'ils sont encore durs (Torque=1) . Il vaut mieux éteindre la carte électronique avant.
  - Le fichier *test_real_draw_forms.py* permet de lancer un test sur le robot tel que l'effecteur final dessine un cercle. Il faudra vous inspirer des fonctions qui permettent de commander les moteurs pour faire les tests sur le robot réel.
- **Détecter l'orientation du cube**
  - Coller un ou plusieurs markeurs ArUco sur les surfaces du cube.
  - Créer un réseau de neurones qui détecte les coins du cube et utilise un calcul mathématique pour obtenir l'orientation :
    - PoseCNN.
    - DenseFusion.
  - Utiliser des méthodes de vision par ordinateur classiques :
    - Calcul à partir du nuage de points avec la méthode Principal Component Analysis (PCA).
    - Détection de l'orientation à partir des 6 plans avec RANSAC.

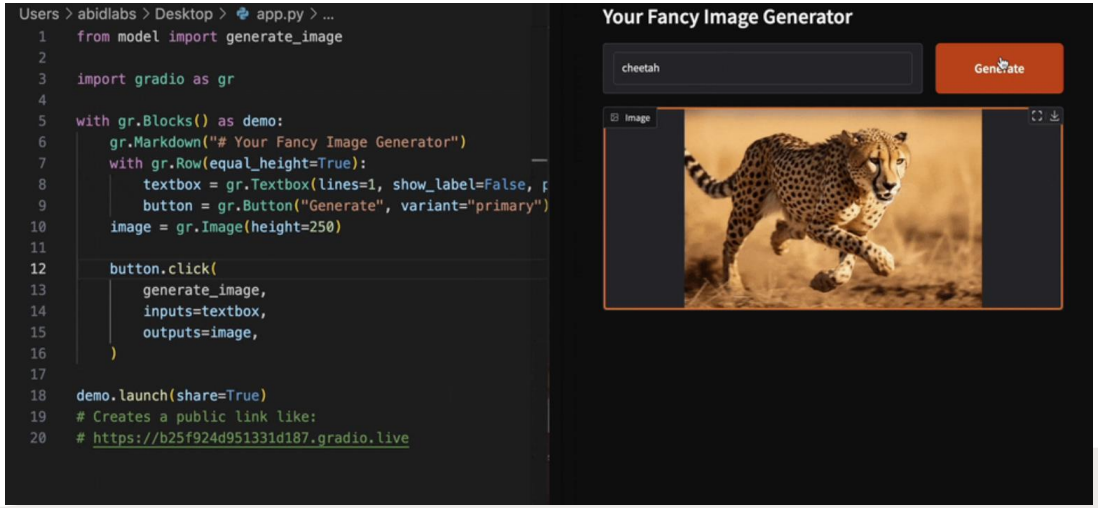




# Étape 9 : Création de l'interface graphique

- **Objectif** : créer une interface graphique qui permet de piloter le projet.
- **Les outils utilisés ou le design reste à votre convenance** : (exemple d'outils)
  - Streamlit : https://streamlit.io/
  - Gradio : https://www.gradio.app/guides/quickstart



# Étape 10 : Présentation & Démonstration

- **Soutenance de la SAE – 26 mars 2026 à partir de 13h00** :
  - Durée : 20 à 25 minutes par groupe
    - 10 minutes de présentation (incluant 1 à 2 minutes de démonstration)
    - 10 minutes de questions par le jury (incluant des questions sur votre code)
  - **Participation** chaque membre du groupe doit parler, même brièvement (1 minute minimum) pour présenter son travail dans le cadre de la SAE. Synchronisez-vous pour ne pas perdre de temps.
  - **Attention** : prévoyez des vidéos de substitution au cas où la démonstration ne pourrait pas se faire.
- **Document d'auto-évaluation à rendre la veille, le 25 mars** (1 page maximum, recto) :
  - Noter chaque membre du groupe (vous inclus) sur 4 points selon votre estimation du travail réalisé.
  - Objectif : viser une moyenne de 3/4 pour l'équipe.
  - Préciser les tâches accomplies, les défis rencontrés, etc.
  - Ce document sera pris en compte par le jury dans l'évaluation globale.
- **Évaluation technique et compréhension** (sur 16 points) :
  - 2 pts -> Préparation du matériel & de la perception.
  - 2 pts -> Environnement de simulation.
  - 2 pts -> Études des algorithmes de DRL.
  - 4 pts -> entraînement, test et analyse des résultats en simulé.
  - 4 pts -> passage Sim-to-real et analyse des résultats sur le robot réel.
  - 2 pts -> Interface graphique.

# Bon Courage !